



INSTITUT
D'ÉLECTRONIQUE
ET D'INFORMATIQUE
GASPARD-MONGE

Algorithmique et Programmation 1

Lundi 1^{er} décembre 2025

L1 Mathématiques - L1 Informatique
Semestre 1

Un complément sur les fichiers

Itérables (encore une fois)

Définition

Un objet dont on peut parcourir les éléments.

- Les dictionnaires `dict`
- Les intervalles `range`
- Les listes `list`
- Les chaînes de caractères `str`
- Les tuples `tuple`
- Les ensembles `set`
- Les fichiers `file`

Un système de sauvegarde

Quand on quitte un programme (au hasard : PikTok), les données en mémoire de Python sont perdues. Comment faire ?

Un système de sauvegarde

Quand on quitte un programme (au hasard : PikTok), les données en mémoire de Python sont perdues. Comment faire ?

Les sauvegarder dans un fichier !

→ `fichier.write(chaine)` écrit `chaine` dans `fichier`.

Si les données sont sous forme d'une chaîne de caractères texte :

```
def sauvegarde(texte, fichier):  
    f = open(fichier, 'w')    # w = write  
    f.write(texte)            # écrire dans le fichier  
    f.write('\n')             # retour à la ligne  
    f.close()                 # fermer le fichier
```

Un système de sauvegarde

Mais après il faut lire !

→ `fichier.read()` renvoie le contenu du fichier `fichier` sous forme de chaîne de caractères.

Un système de sauvegarde

Mais après il faut lire !

→ `fichier.read()` renvoie le contenu du fichier `fichier` sous forme de chaîne de caractères.

```
def recuperer_sauvegarde(fichier):  
    f = open(fichier, 'r') # r = read  
    texte = f.read()  
    # Pour retirer le retour à la ligne  
    texte = texte.strip('\n')  
    f.close()  
    return texte
```

Fermeture automatique

C'est pénible de devoir penser à fermer le fichier à chaque fois.
Python peut le faire automatiquement pour nous :

```
def sauvegarde(texte, fichier):  
    # Mieux :  
    with open(fichier, 'w') as f: # w = write  
        f.write(texte)  
        f.write('\n') # retour à la ligne  
    # le fichier est fermé automatiquement
```

Idem quand on veut le lire :

```
def recuperer_sauvegarde(fichier):  
    # Mieux :  
    with open(fichier, 'r') as f: # r = read  
        texte = f.read()  
    return texte
```

→ Voir aussi lire_fichier.py et ecrire_fichier.py.

Problème : comment stocker les données sous formes de chaîne de caractères ?

Exemple 1 : stocker des pseudos

Objectif : stocker la liste des pseudos des joueurs de PikTok...

→ Sur Thonny : `sauvegarde_pseudos.py`.

Problème : comment stocker les données sous formes de chaîne de caractères ?

Exemple 1 : stocker des pseudos

Objectif : stocker la liste des pseudos des joueurs de PikTok...

→ Sur Thonny : `sauvegarde_pseudos.py`.

... et la récupérer !

Exemple 2 : stocker une liste

Objectif : stocker une liste et la récupérer.

→ Sur Thonny : `stocker_liste.py`.

Exemple 2 : stocker une liste

Objectif : stocker une liste et la récupérer.

→ Dès que les données se compliquent, ça devient un cauchemar...
Qu'est-ce que ça va être pour PikTok ???!!!

Exemple 2 : stocker une liste

Objectif : stocker une liste et la récupérer.

→ Dès que les données se compliquent, ça devient un cauchemar...
Qu'est-ce que ça va être pour PikTok ???!!!

JSON

Heureusement, les développeurs Python on pensé à nous !

(et à eux-mêmes)

→ JSON permet de stocker et de récupérer n'importe quel objet Python.

`json.dump(objet, fichier)` stocke objet dans fichier.

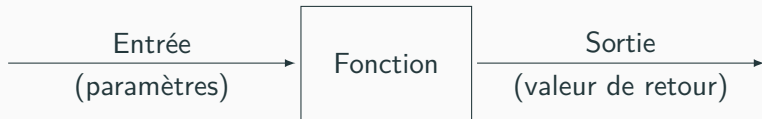
`json.load(fichier)` récupère l'objet stocké dans fichier.

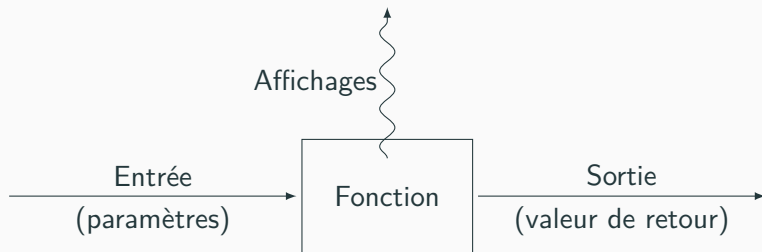
Retour sur les fonctions

Fonction

En informatique, une fonction est :

- Un morceau de programme
- Portant en général *un nom*
- Prenant un ou plusieurs *paramètres* (ou zéro)
- Renvoyant un résultat (la plupart du temps)





Définition et appel de fonction

Définir une fonction

La syntaxe pour définir une fonction est :

```
def nom_fonction(parametre_1, ..., parametre_n):  
    # corps de la fonction  
    # utilisant parametre_1, ..., parametre_n  
    ...  
    # peut renvoyer un résultat :  
    return resultat
```

Définition et appel de fonction

Définir une fonction

La syntaxe pour définir une fonction est :

```
def nom_fonction(parametre_1, ..., parametre_n):  
    # corps de la fonction  
    # utilisant parametre_1, ..., parametre_n  
    ...  
    # peut renvoyer un résultat :  
    return resultat
```

Appeler une fonction

On peut ensuite *appeler* la fonction `nom_fonction` avec le code :

```
resultat = nom_fonction(expression_1, ..., expression_n)
```

Examples

Dessiner un rectangle

Dessiner un *rectangle* fait du caractère caractere :

Dessiner un rectangle

Dessiner un *rectangle* fait du caractère caractere :

```
def dessine_rectangle(m, n, caractere):  
    for i in range(m):  
        for j in range(n):  
            print(caractere, end = ' ')  
        print()
```

Composition de fonctions

On peut appeler une fonction dans une fonction ! (et ainsi de suite)

Composition de fonctions

On peut appeler une fonction dans une fonction ! (et ainsi de suite)

Dessiner un rectangle : variante

```
def dessine_ligne(n, caractere):  
    for j in range(n):  
        print(caractere, end = ' ')  
    print()  
  
def dessine_rectangle(m, n, caractere):  
    for i in range(m):  
        dessine_ligne(n, caractere)
```

Composition de fonctions

On peut appeler une fonction dans une fonction ! (et ainsi de suite)

Dessiner un rectangle : variante

```
def dessine_ligne(n, caractere):  
    for j in range(n):  
        print(caractere, end = ' ')  
    print()  
  
def dessine_rectangle(m, n, caractere):  
    for i in range(m):  
        dessine_ligne(n, caractere)
```

Question

→ Qu'est-ce qui empêche une fonction de s'appeler *elle-même* ?

Dessiner un rectangle : deuxième variante

```
def dessine_ligne(n, caractere):  
    for j in range(n):  
        print(caractere, end = '')  
    print()  
  
def dessine_rectangle(m, n, caractere):  
    if m > 0:  
        dessine_ligne(n, caractere)  
        dessine_rectangle(m-1, n, caractere)
```

La récursivité

Fonction réursive

→ Une fonction qui s'appelle elle-même.

Exemple : factorielle

$$0! = 1$$

$$n! = n \times (n - 1)!$$

Fonction réursive

→ Une fonction qui s'appelle elle-même.

Exemple : factorielle

$$0! = 1$$

$$n! = n \times (n - 1)!$$

```
def fact(n):  
    if n == 0:  
        return 1  
    else:  
        return n * fact(n-1)
```

Exemple : dessiner une spirale

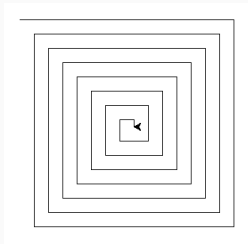


Figure 1 : Une spirale

Pour dessiner une spirale de côté $c > 0$:

1. On trace un trait de longueur c
2. On tourne le stylo de 90 degrés
3. On trace une spirale de côté $c - d$, où d est l'écart entre un côté de la spirale et le suivant.

Teaser : l'arbre d'Auto-Warhol

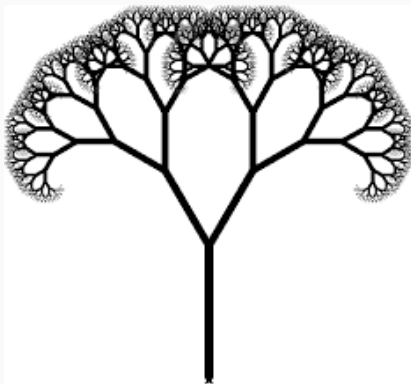


Figure 2 : Un arbre

Fonction récursive

→ Une fonction qui s'appelle elle-même.

```
def fact(n):  
    return n * fact(n-1)
```

Fonction récursive

→ Une fonction qui s'appelle elle-même.

```
def fact(n):  
    return n * fact(n-1)
```

Attention !

Pour qu'elle termine, une fonction récursive doit avoir un ou plusieurs *cas de base*.

(comme une boucle `while` doit avoir une condition d'arrêt)

Appels récursifs

```
def fact(n):  
    if n == 0:  
        return 1  
    else:  
        return n * fact(n)
```

Appels récursifs

```
def fact(n):  
    if n == 0:  
        return 1  
    else:  
        return n * fact(n)
```

Attention !

Pour qu'elle termine, une fonction récursive doit s'auto-appeler sur des arguments *plus simples*.

(comme une boucle **while** qui dépend de `n` doit modifier `n`, sinon elle ne termine pas)

Bonus

Exemple : l'arbre d'Auto-Warhol

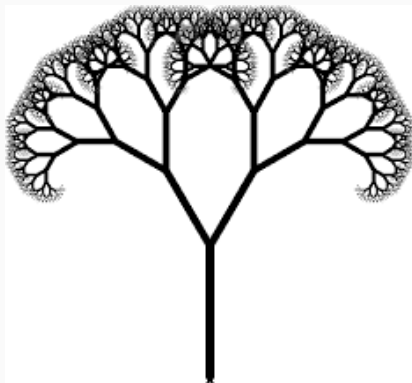


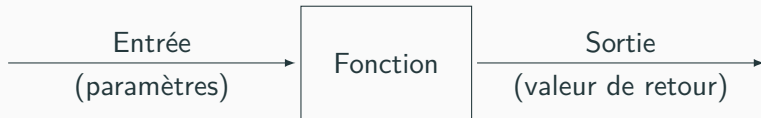
Figure 3 : Un arbre

Exemple : l'arbre d'Auto-Warhol

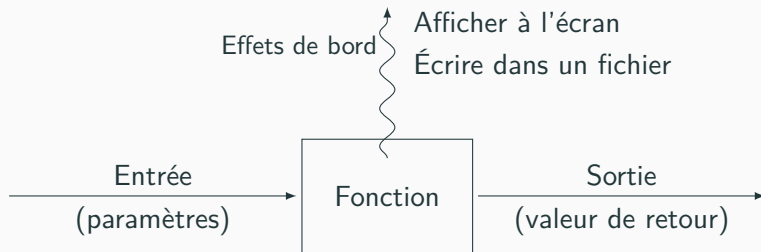
Pour dessiner un arbre dont le tronc mesure L (si $L > 1$) :

1. On trace un trait de longueur L
2. On tourne le stylo de 30 degrés vers la gauche
3. On trace un arbre de tronc $\frac{2}{3}L$
4. On tourne le stylo de 30 degrés vers la droite pour le remettre en position
5. On tourne le stylo de 30 degrés vers la droite
6. On trace un arbre de tronc $\frac{2}{3}L$
7. On tourne le stylo de 30 degrés vers la gauche pour le remettre en position
8. On ramène le stylo à sa place en traçant un trait de longueur L à l'envers

Suppléments sur les fonctions



Fonctions



Fonctions

